

EECS4302 Compilers and Interpreters Summer 2025

Project Report

1 Input Languages	1
1.1 Structure of a Program	1
1.2 List of Advanced Programming Features	5
1.2.1 Feature 1: Class/Module Object Declarations & Inheritance	5
1.2.2 Feature 2: Multi-dimensional Arrays	6
1.2.3 Feature 3: Built-in Array Operations – append() & insert()	7
1.2.4 Feature 4: Function Declarations and Invocations	8
1.2.5 Feature 5: Iterations (Bounded and Unbounded)	9
1.2.6 Feature 6: print statements	10
2 Output Structure	11
HTML Output Generation	11
3 Justification of Output	14
3.1 Justification for Variable Declaration and Type Consistency	14
3.2 Justification for Operation Type Checking	14
3.3 Justification for Function Declarations and Calls	14
3.4 Justification for Loop Type Checking	15
3.5 Justification for Object Support, Inheritance, and Polymorphism	15
3.6 Justification for Array Type Checking	15
4 Summary of Submitted Examples	16
5 Miscellaneous Features	19
5.4. Extensible Architecture	21
6 Limitations	22
CONTRIBUTIONS:	24

1 Input Languages

1.1 Structure of a Program

The following example shows what a typical input program for the compiler looks like without errors (**Note**: Class names always start with an uppercase letter, while variable and function names always start with a lowercase letter. Data types are always fully capitalized.):

```
// class/module declarations
class Parent {
    // variable declarations
    NUMBER v = 10;
    STRING v2 = "base";
    BOOL flag;
    // data structures initialized with literals
    NUMBER[][] num_arr = [[0, 0, 0],
```

```

        [0, 0, 0],
        [1, 2, 3]];
STRING[] str_arr = ["", "", [[], []], []];
BOOL[][] bool_arr = [[false, true], [true]];
bool_arr[0] = [true];
bool_arr[1] = [false];
BOOL[][] bool_arr2;
bool_arr2[0] = [true];
bool_arr2[1] = [false];

// empty data structures
BOOL[] empty_bool_arr;
BOOL[][] bool_arr3 = [[true], [false], [[], [], []]];
bool_arr3[0] = empty_bool_arr;

// function/method declarations
fun foo(a: NUMBER): NUMBER {
    // variable references and arithmetic expressions
    return a * 2;
}
}
// object-oriented features (inheritance)
class Child inherits Parent {
    STRING name = "derived";
    fun bar(): VOID {
        // function/method invocations
        print("Hello from B");
    }
}
fun compute(x: NUMBER, y: NUMBER): NUMBER {
    // grouping and arithmetic expressions
    return ((x + y) * (x - y)) / 2;
}
fun testOps(): VOID {
    // more variable declarations
    NUMBER a = 5;
    NUMBER b = 3;
    BOOL t = true;
    BOOL f = false;
    STRING s1 = "foo";
    STRING s2 = "bar";
    // all possible arithmetic expressions
    NUMBER num_example = a ^ b;

```

```

num_example = a % b;
num_example = a * b;
num_example = a / b;
num_example = a + b;
num_example = a - b;
// relational expressions
BOOL bool_example = a == b;
bool_example = t == f;
bool_example = s1 == s2;
bool_example = a != b;
bool_example = a < b;
bool_example = a > b;
bool_example = a <= b;
bool_example = a >= b;
// logical expressions
bool_example = not t;
bool_example = t and f;
bool_example = t or f;
bool_example = t -> f;
STRING joined = s1 & s2;
STRING s3 = "This number is: " & 1;
joined = joined & 1 & true & 2;
NUMBER g = ((a + b) * (a - b)) ^ 2;
// more data structures -- simplified sizes
NUMBER[][] arr1 = [[0, 0],
                   [0, 0]];
STRING[] arr2 = ["", ""];
BOOL[] arr3 = [false, false];

// append operation with array's of primitive data types
append(arr1, [10, 20]);
append(arr2, "what");
append(arr3, true);

append([1, 2, 3], 1);
append([[true, false], [false]], [true]);
append(["str1"], ["str2", "str3"], ["str4"]);

// insert operation with array's of primitive data types
insert(arr1, 1, [1]);
insert(arr2, 0, "insert");
insert(arr3, 2, false);

```

```

insert([1, 2, 3], 1, 1);
insert([[true, false], [false]], 0, [true]);
insert(["str1"], ["str2", "str3"], 2, ["str4"]);

// array index access
arr1[1][1] = 100;
arr2[0] = "hello";
compute(a, b);
}
fun controlStructures(): VOID {
    NUMBER i = 0;
    // unbounded iteration
    while (i < 5) {
        print(i);
        i = i + 1;
    }
    // bounded iteration
    for (NUMBER j = 0; j < 3; j = j + 1;) {
        print(j);
    }
    // conditionals
    if (true) {
        print("Yes");
    } else {
        print("No");
    }
}
fun main(): VOID {
    // object-based features (object creation, method calls, dot
notation)
    Child obj2 = new Child;
    // polymorphism & object arrays
    Parent obj = obj2;
    Parent[] arrObj = [obj, obj2];

    // append operation with array's of class data types
    append(arrObj, obj);
    insert(arrObj, 0, obj2);
    obj.foo(10);
    // calling operations from non-class functions
    testOps();
    controlStructures();
    // accessing fields and functions from class/module objects

```

```

print(obj.foo(10));
print(obj.v & " " & obj.v2 & " " & obj.flag);
print(obj.num_arr[1][2]);
print(obj.str_arr[0]);
}

```

1.2 List of Advanced Programming Features

1.2.1 Feature 1: Class/Module Object Declarations & Inheritance

Syntax:

```

prog: (classDecl | funcDecl | decl | stmt)+ EOF
    ;

classDecl:
    'class' CLASSNAME ('inherits' CLASSNAME)? '{' (decl | funcDecl | stmt)* '}'

decl: CLASSNAME ID '=' 'new' CLASSNAME ';'          # ObjectInitializedDeclaration
    ;

stmt: (ID | ID '.' ID) '=' expr ';'                # AssignmentStatement
    ;

```

When a class is decelerated using the 'class' keyword, the class has the option to inherit from a base class using the 'inherits' keyword. The #ObjectInitializedDeclaration allows the creation of class objects while #AssignmentStatement allows access to the class's fields and methods using '.' notation. Its body consists of variable decelerations, functions, & programming statements. When the object of a class is made, if it has inherited from another class, it gets access to all its parent class's fields and functions using the '.' notation.

Example: In the following example, since class B inherits from class A, class B can also access field x and function fun using '.' notation.

```

class A {
    NUMBER x = 10;
    fun foo(a: NUMBER): NUMBER {
        return a / 10;
    }
}

class B inherits A {
    fun bar(): VOID {
        print("Hello from B");
    }
}

```

```
B d = new B;
print(d.foo(10));
```

Cross-reference:

- **full_test_suite1.txt**: Classes A, B, & Main are declared, and class B inherits class A & class Main inherits class B. The Main class makes class A & class B objects.
- **full_test_suite2.txt**: The Utils & Main classes are declared such that the Main class inherits from the Utils class.
- **test1_declarations.txt**: Class Parent is declared
- **test2_expressions.txt**: TestExpressions class is declared
- **test3_conditionals.txt**: TestConditionals class is declared
- **test4_function_calls.txt**: FunctionTest class is declared
- **test5_inheritance_visibility.txt**: Base, Child, & GrandChild classes are declared. The Child class inherits from the Base class, while the GrandChild class inherits from the Child class. The GrandChild class makes a Base class object.
- **test6_loops.txt**: LoopTest class is declared
- **test7_objects_access.txt**: Animal, Dog, & ObjectTest classes are declared. The Dog class inherits from the Animal class. The ObjectTest class makes a Dog class object.
- **test8_arrays.txt**: ArrayTest class is declared.

1.2.2 Feature 2: Multi-dimensional Arrays

Syntax:

```
decl: type ID '=' expr ';'           # PrimitiveInitializedDeclaration
    | type ID ';'                   # PrimitiveUninitializedDeclaration

stmt: expr '[' expr ']' '=' expr ';' # ArrayAssignment

type: (INT_TYPE | BOOL_TYPE | STRING_TYPE | VOID_TYPE | CLASSNAME) ('[' ' '])*
    ;

expr: expr '[' expr ']'              # ArrayAccess
    | '[' exprList? ','? ']'        # ArrayLiteral

exprList: expr (',' expr)*
    ;
```

Arrays can be declared with one or more dimensions (e.g., new NUMBER[3][4]). They can be either of a primitive data type or a class data type, but making a VOID_TYPE array will result in a semantic error. # **ArrayAccess** allows any one of its dimensions to be accessed to assign or use the values associated with a specific index, while # **ArrayAccess** is used to declare the array with an array literal in the form of (' = [expr, expr, ...]).

Cross-reference:

- **full_test_suite1.txt**: The Main class declares primitive & class object data type arrays with 1 or 2 dimensions.

- **full_test_suite2.txt**: The Utils class declares primitive data types of 1-and 2-dimensional arrays.
- **test1_declarations.txt**: The parent class declares primitive data types of 1-and 2-dimensional arrays.
- **test8_arrays.txt**: The ArrayTest class declares primitive data types of 1-and 2-dimensional arrays.

1.2.3 Feature 3: Built-in Array Operations – append() & insert()

Syntax:

```

stmt:  appendStmt                                # AppendStatement
      |  insertStmt                              # InsertStatement
      ;

appendStmt: 'append' '(' expr ',' expr ')' ';' ;
insertStmt: 'insert' '(' expr ',' expr ',' expr ')' ';' ;

```

The append() and insert() functions are built-in operations for inserting values into an array. The functions are structured such that:

- **append(array, value)**; adds the value to the end of the array
- **insert(array, index, value)**; inserts the value at a specified index, pushing any subsequent elements to the right.

The ‘array’ is the target array, ‘index’ is the number representing the index where a value is to be inserted into the array, and ‘value’ is the value being inserted into the array.

Example:

```

NUMBER[] myArray = [1, 2, 3];
append(myArray, 42);           // Appends 42 to the end of myArray
insert(myArray, 1, 99);        // Inserts 99 at index 1 in myArray

```

Cross-reference:

- **full_test_suite1.txt**: The Main class uses both the append() & insert() operations to insert elements into arrays.
- **full_test_suite2.txt**: The Utils class implements both the append() & insert() operations to insert elements into arrays

1.2.4 Feature 4: Function Declarations and Invocations

Syntax:

```

prog: (classDecl | funcDecl | decl | stmt)+ EOF
      ;

stmt:
  | returnStmt                                # ReturnStatement
  | funcCall ';'                              # FunctionCallStatement

funcCall
  : ( ID | ID '.' ID ) '(' argList? ')'
  ;

```

```

argList
    : expr (',' expr)*
    ;
// === FUNCTION DECLARATION ===
funcDecl
    : 'fun' ID '(' paramList? ')' ':' type block
    ;

paramList
    : param (',' param)*
    ;

param: ID ':' type
    ;

block: '{' (decl | stmt)* '}'
    ;

```

The compiler supports the declaration of functions and methods, and allows for their invocation both globally (i.e., not associated with a class) and as object methods using '.' notation as specified in `funcCall`. The functions can take 0 or more parameters of either primitive or class data type as specified in `paramList`. A function can have a return type of NUMBER, BOOLEAN, STRING using the 'return' keyword as specified # `ReturnStatement` or no output value returned (and thus no 'return' keyword) if the type is VOID if nothing is returned. The function's body can be composed of variable declarations and programming statements as specified by `block`.

Example:

```

class FunctionObject {
    fun printData(x: STRING, y: NUMBER): VOID {
        print(x & y);
    }
}

fun square(x: NUMBER): NUMBER {
    return x * x;
}

```

```

FunctionObject obj = new FunctionObject;
obj.printData("data: ", square(8));

```

Cross-reference:

- **full_test_suite1.txt**: Class A declares functions that are used by both the B & Main classes, which inherit from class A.
- **full_test_suite2.txt**: The Utils class declares functions that are used both internally and by its child class, Main.
- **test1_declarations.txt**: Cass Parent makes functions.

- **test4_function_calls.txt**: The FunctionTest class makes some functions that it uses to output values for variable decelerations.
- **test5_inheritance_visibility.txt**: The Base class creates a single function that's used by the Child & GrandChild classes, which inherit from the Base class.
- **test6_loops.txt**: The LoopTest class declares a VOID_TYPE function that holds all the code that will be run.
- **test7_objects_access.txt**: The Animal class makes a function that's class called by a Dog class object, which inherited from the Animal class.

1.2.5 Feature 5: Iterations (Bounded and Unbounded)

Syntax:

```

stmt:
    | whileStmt                                # WhileLoopStatement
    | forStmt                                  # ForLoopStatement

whileStmt: 'while' '(' expr ')' block
    ;
forStmt: 'for' '(' forInt? forCond? stmt? ')' block
    ;
forInt: (decl | stmt)
    ;
forCond: expr ';'
    ;
// === BLOCK ===
block
    : '{' (decl | stmt)* '}'
    ;

```

The compiler can support unbounded while loops, which take a boolean expression in their header as shown in **whileStmt**. The compiler also supports bounded for loops, along with said for loop's initializer, condition, and update sections to specify the duration of the bounded loop as shown in **forStmt**. The **forInt** grammar initializes a pre-existing or declared numerical value. The **forCond** grammar is just a boolean expression that determines whether the loop iterates. Finally, the **stmt** is the expression where the value initialized in **forStmt** is updated. The 'block' notation represents the inner body of both loops, which is composed of variable declarations and programming statements that can even allow for nested loops.

Example:

```

for (NUMBER i = 0; i < 5; i = i + 1;) {
    while (x > 0) {
        print(x);
        x = x - 1;
    }
}

```

```

    }
    print(i);
}

```

Cross-reference:

- **full_test_suite.txt:** The Main class uses both while & for loops.
- **test6_loops.txt:** The LoopTest class uses both while & for loops.

1.2.6 Feature 6: print statements

Syntax:

```

stmt: printStmt                                     # PrintStatement
    ;

printStmt: 'print' '(' expr ')' ';'
    ;

```

print statements are statements that consist of the keyword 'print', followed by a parenthesized string value, and terminated with a semicolon as specified by **printStmt**. The statement doesn't output the value of its expression on the console but serves as a representation of the print statement that's commonly used by most programming languages.

Example:

```

print("Hello, world");
NUMBER a = 5;
NUMBER b = 7;
print(a + b);    // Outputs 12

```

Cross-reference:

- **full_test_suite.txt:** The Main class uses print statements
- **full_test_suite2.txt:** The Main class uses print statements
- **test3_conditionals.txt:** The TestConditionals declares print statements in its conditional statements
- **test6_loops.txt:** In the LoopTest class, the while & for loops both have print statements
- **test7_objects_access.txt:** The for loop in Animal class has a print statement

2 Output Structure

The outputted HTML file is used to help the user understand the results of semantic analysis and type checking that the compiler does with the input program for any errors that may occur. It consists of two main components:

HTML Output Generation

Each time a source file is compiled, the compiler generates an HTML report summarizing the semantic analysis. This file is automatically saved to disk and can be opened in any web browser.

The report is divided into three main visual panels:

- **Left Panel: Semantic Error Report**
 - All semantic errors are displayed in clearly marked red boxes.
 - Each box includes the error message, class name, line number, character position, and the relevant context.
 - If no errors exist, the panel explicitly states that no semantic issues were found.
- **Right Panel: Class Structure Overview**
 - Each class is displayed in a separate visual card.
 - Cards include:
 - **Class name**, including any parent class if inheritance applies.
 - **Variables** are listed with their declared types.
 - **Functions** are listed with their return type and full parameter type signature.
 - Inherited members are stripped from the display to avoid redundancy. Only class-local declarations appear.
- **Bottom Panel: Annotated Source Code View**
 - The full source code is displayed with line numbers.
 - Any line associated with a semantic error is highlighted with a red background and border.
 - The display preserves formatting and syntax via a monospace font and whitespace-preserving layout.

Semantic Errors

ArrayLiteral: Inconsistent element types in array literal

Class: ArrayTest, Line: 6, Char: 19

PrimitiveInitDecl: Type mismatch: ERROR expected for variable bad

Class: ArrayTest, Line: 6, Char: 4

ArrayAssign: Type mismatch: assigning STRING to array element of type NUMBER

Class: ArrayTest, Line: 9, Char: 4

ArrayAssign: Cannot index non-array type: NUMBER

Class: ArrayTest, Line: 13, Char: 4

PrimitiveInitDecl: Type mismatch: NUMBER[] expected for variable badDepth

Class: ArrayTest, Line: 16, Char: 4

ArrayLiteral: Inconsistent element types in array literal

Class: ArrayTest, Line: 19, Char: 25

PrimitiveInitDecl: Type mismatch: ERROR expected for variable badNest

Class: ArrayTest, Line: 19, Char: 4

Class: ArrayTest

Variables

NUMBER x
NUMBER[][] badNest
NUMBER[] bad
NUMBER[][] matrix
NUMBER[][] badDepth
NUMBER[] nums

Functions

None

Source Code

```
1 class ArrayTest {  
2     NUMBER[] nums = [1, 2, 3];  
3     NUMBER[][] matrix = [[1, 2], [3, 4]];  
4  
5     // ERROR: mixed types  
6     NUMBER[] bad = [1, true]; // ERROR: BOOL in NUMBER[]  
7  
8     // ERROR: bad index assign  
9     nums[1] = "hi"; // ERROR: STRING into NUMBER[]  
10  
11    // ERROR: indexing non-array  
12    NUMBER x = 5;  
13    x[0] = 1; // ERROR: scalar used as array  
14  
15    // ERROR: assigning 1D to 2D  
16    NUMBER[][] badDepth = nums; // ERROR: depth mismatch  
17  
18    // ERROR: type in nested array  
19    NUMBER[][] badNest = [[1, 2], ["oops"]]; // ERROR: STRING in NUMBER[][]  
20 }
```

Type Checking Criteria

The compiler applies the following type checking rules to determine semantic errors:

Variable Usage

- A variable used in an expression must be declared beforehand
- Variables must only be assigned expressions of the same data type
- NUMBER, STRING, & BOOL data types are primitive data types representing numerical, string, & boolean values, while CLASSNAME data types represent class data types.

Loop Constructs

- The header expression of a while loop must be of data type BOOL
- For-loops have 3 optional sections:
 - Initialization: A NUMBER expression that assigns a value to a pre-existing or declared NUMBER variable.
 - Condition: A BOOL expression that allows the for loop to iterate if it's true.
 - Update Expressions: A NUMBER expression that assigns an updated value to a NUMBER variable.

Function Declarations and Calls

- A function must be declared before it is called
- Each argument in a function call must match the expected data type declared in the function's signature
- A function must return a value of the declared return type. If the declared return type is VOID, they can't use 'return'.

- The 'return' keyword must return an expression.

Classes and Object-Oriented Features

- A class must be declared before it is used for object instantiation or inheritance
- The fields & methods accessed via a class object must exist in the referenced class's definition.
- A subclass inherits all fields and methods from its superclass.
- A Subclass object can be upcasted by being assigned to a superclass variable, but not the other way around.
- Calling fields or methods without a class object is only valid if they're declared outside of a class and are called anywhere in the program after their declaration

Arithmetic and Logical Expressions

- Arithmetic operators (+, -, *, /, %, ^) must only be applied to NUMBER type expressions.
- Relational (<, >, <=, >=) operators must be used between 2 NUMBER or BOOL expressions (the expressions must be of matching data type)
- Logical (and, or, not, ->) operators must be used between 2 BOOL expressions.

Array Rules

- Arrays must be declared before they are used
- An array can't be assigned an array with a different data type or a different number of dimensions
- All expressions assigned to an array index must match the array's data type.
 - If an array's data type is an object class, each element assigned to the array must be either an instance of the same class as the array's declared type or an instance of a subclass of that class.
- All forms of empty array literals (i.e. [], [[]], [[[]], []], etc...) are treated as the same special "EMPTY_ARRAY" type, regardless of how many nested brackets they have.
 - Empty arrays can only be put in arrays where the type can be inferred from other non-empty array elements. (i.e. BOOL[] bools2 = [true, [[]];)
 - Empty arrays can't be assigned to an array index or be used to initialize an array variable when it's declared, since the system treats them as an ambiguous EMPTY_ARRAY type, which doesn't match any declared data type or dimensionality, leading to type mismatch errors
 - You can still declare an empty array by specifying the type and dimensions, without assigning a value (i.e. NUMBER[] numArr; BOOL[][] boolMatrix;)
- The expression used to index an array must be of data type NUMBER.
- Values can't be assigned to the index of an array with an empty literal (i.e., has no NUMBER, BOOL, STRING, or CLASSNAME data type values in it).
- For array operations 'append(array, value);' & 'insert(array, index, value);', the data types of their arguments are:
 - **array**: An array data type of any type
 - **index**: A NUMBER expression
 - **value**: A primitive data type is the 'array' specified is a 1-dimensional array or a multidimensional array if 'array' is also a multidimensional array, whose dimension count depends on the dimension count of 'array'

3 Justification of Output

3.1 Justification for Variable Declaration and Type Consistency

Our compiler enforces that every variable must be declared before use, and that the type of a variable matches the type of any value assigned to it. For example, assigning a `STRING` to a `NUMBER` variable or using an undeclared identifier is flagged as an error.

Cross-reference:

- In `full_test_suite1.txt`, the line `NUMBER err3 = undeclaredVar;` triggers a semantic error for using an undeclared variable.
- In the same file, `NUMBER err4 = true;` is flagged as a type mismatch.
- In `test1_declarations.txt`, `STRING badInit = 123;` produces a similar type mismatch.
- The output HTML file correctly highlights each of these cases and associates the error with the variable type consistency rule.

3.2 Justification for Operation Type Checking

The compiler enforces operand type compatibility for arithmetic, logical, and relational operators. For example, adding a `NUMBER` to a `BOOL` is invalid, and using a non-boolean operand in a logical expression is flagged.

Cross-reference:

- In `test2_expressions.txt`, `BOOL badAdd = true + 1;` correctly produces a type error for invalid arithmetic.
- `BOOL invalidLogic = a and b;` is flagged because logical operators must operate on `BOOL` values.
- In `full_test_suite1.txt`, `NUMBER err5 = 4 + false;` and `BOOL err6 = 1 or true;` show the same kind of invalid operations.
- The output HTML files for these tests point out exactly why these operations are invalid and reference the types involved.

3.3 Justification for Function Declarations and Calls

Functions are statically checked against their declared signatures. Argument count and types must match the declaration, and the return type must align with the context in which the function is called.

Cross-reference:

- In `test4_function_calls.txt`, `NUMBER tooFewArgs = multiply(10);` and `NUMBER badArgs = multiply(true, "oops");` produce errors for incorrect argument counts and types.

- Also in `test4_function_calls.txt`, `NUMBER wrongType = alwaysTrue();` causes an error because a `BOOL` function is being assigned to a `NUMBER` variable.
- In `full_test_suite1.txt`, `NUMBER err1 = obj.double("string");` and `NUMBER err7 = double(1, 2);` show invalid argument types and excess parameters respectively.
- The output files clearly indicate the mismatch in the call signature vs. declaration, fulfilling this type checking criterion.

3.4 Justification for Loop Type Checking

All loop conditions (`while` and `for`) must evaluate to `BOOL`. Initialization and update expressions are also type-checked to ensure proper numeric usage.

Cross-reference:

- In `test6_loops.txt`, `while ("bad")` and `for (NUMBER k = 0; 3; k = k + 1;)` generate type errors because the conditions are not boolean.
- These errors are accurately marked in the output HTML file, confirming correct enforcement of loop condition typing.

3.5 Justification for Object Support, Inheritance, and Polymorphism

Our language supports object-oriented constructs including class-based inheritance, method and field access, and polymorphism through upcasting. All access and instantiation are type checked for correctness.

Cross-reference:

- In `test5_inheritance_visibility.txt`, valid field access (`getBase()`) and invalid field access (`obj.notReal`) both appear.
- `test7_objects_access.txt` contains valid inherited access (`d.alive`) and invalid member accesses (`d.age` and `d.bark()`), which are flagged as semantic errors.
- In `full_test_suite1.txt`, `A baseRef = new B;` demonstrates polymorphic assignment and is used correctly for method calls.
- The output confirms that the type checker correctly resolves visibility across inheritance hierarchies and catches invalid member access.

3.6 Justification for Array Type Checking

Arrays are statically typed and must contain elements matching their declared base type. Indexing and mutation operations are type-checked, and multi-dimensional arrays require consistent nested types.

Cross-reference:

- In `test8_arrays.txt`, `NUMBER[] bad = [1, true];` is flagged due to type mismatch in array initialization.
- `nums[1] = "hi";` shows invalid assignment to a typed array.
- In `full_test_suite.txt`, examples like `insert(nums, 0, "bad");`, `append(bools, 1);`, and `append(polyArr, "string");` are caught for inserting/adding the wrong type into arrays.
- Other errors such as `nonArr[0] = 1;` and `BOOL badAssign = [true, false];` are also correctly flagged.
- The output files correctly report the specific nature of each violation, confirming full array support and type enforcement.

4 Summary of Submitted Examples

Test cases are created to cover each key feature supported by our programming language, demonstrating its capabilities and error detection. Since there is no need to repeat contents of the submitted working examples, these sections won't include test case input/output and only provide highlights of each test case.

4.1 Example Input 1: test1_declarations.txt

- Demonstrates variable declarations with primitive types (**NUMBER**, **STRING**, **BOOL**) and multi-dimensional arrays.
- Includes function declarations with typed parameters and return values.
- Detects semantic errors such as type mismatches in array literals and invalid variable initializations.
- Identifies return type mismatches in function definitions.

4.2 Example Input 2: test2_expressions.txt

- Demonstrates variable declarations and initialization using arithmetic and relational expressions.
- Validates type checking for arithmetic (**+**), logical (**and**), and relational (**<**, **>**) operators.
- Detects semantic errors including:
 - Addition involving non-number types.

- Logical operations on non-boolean operands.
- Invalid relational comparisons between strings.

4.3 Example Input 3: test3_conditionals.txt

- Demonstrates use of if statements with boolean conditions.
- Validates that conditions in control flow statements must evaluate to boolean values.
- Detects errors caused by using non-boolean expressions in conditional statements.
- Includes examples of correct and incorrect conditional logic.

4.4 Example Input 4: test4_function_calls.txt

- Demonstrates function declarations with typed parameters and return types.
- Shows correct invocation of functions with matching argument types. (**fun functionName(...): TYPE**)
- Detects argument count mismatches in function calls.
- Identifies type mismatches in passed arguments.
- Reports return type mismatches during variable initialization.

*The first 4 examples have demonstrated most of the basic programming features such as:

- **class/module declarations**
- **variable declarations**
- **variable assignments**
- **variable references (i.e., referring to declared variables in expressions)**
- **arithmetic, relational, and logical expressions**
- **conditionals**

The rest of the examples will cover more advanced features

4.5 Example Input 5: test5_inheritance_visibility.txt

- Demonstrates class inheritance and field/method visibility across parent-child relationships.
- Shows proper use of inherited fields and methods from base classes.
- Detects attempts to access undefined or non-existent fields.

- Reports calls to undefined methods.
- Validates type assignments during field initialization.

4.6 Example Input 6: test6_loops.txt

- Demonstrates **while** and **for** loop constructs with proper variable initialization and loop control.
- Validates boolean conditions required in loop statements.
- Detects semantic errors for non-boolean conditions in **while** and **for** loops.

4.7 Example Input 7: test7_objects_access.txt

- Demonstrates object instantiation and access to inherited fields and methods.
- Validates field and method access within class hierarchies.
- Detects errors when accessing undefined fields.
- Identifies calls to undefined methods.

4.8 Example Input 8: test8_arrays.txt

- Demonstrates array declarations with one- and two-dimensional arrays.
- Validates element type consistency within arrays and nested arrays.
- Detects type mismatches in array initializations and assignments.
- Identifies errors when indexing non-array types.
- Reports depth mismatches between arrays of different dimensions.

4.9 Example Input 9: full_test_suite1.txt

The final 2 examples are supposed to be comprehensive, which provide Inheritance, Arrays, Control Flow, Polymorphism, and Semantic Errors

- Demonstrates class inheritance, function declarations, and polymorphism.
- Covers array operations: creation, indexing, insertion, and appending.
- Includes control structures: loops and conditionals.
- Detects semantic errors such as function call mismatches, undefined fields/methods, type mismatches, invalid array operations, and variable redeclarations.

4.10 Example Input 10: full_test_suite2.txt

- Demonstrates complex expressions including arithmetic, logical operators, string concatenation, and 2D array access.
- Shows function declarations with multiple parameters and nested expressions.
- Covers dynamic array insertions and appends.
- Detects semantic errors such as type mismatches in operations, incorrect function argument counts, invalid logical and arithmetic operations, and division by zero.
- Includes use of inheritance with expressions and control structures.

5 Miscellaneous Features

5.1. Command-Line Flags

The compiler now accepts optional flags to control output behavior:

- `--html`: Generates an HTML semantic error report.
- `--console`: Prints semantic errors directly to the console.

These flags can be used independently or together. If no flags are provided, the compiler runs silently unless errors are encountered.

To test input files, open a terminal **inside** the project folder where **my-compiler.jar** is located, and run:

```
java -jar my-compiler.jar input1.txt --console --html
```

Step

Console output of full_test_suite2.txt:

```
Semantic errors found:
Error [AdditionContext] in class 'Utils' at line 37, char 17: Addition only works for numbers
Error [PrimitiveInitDecl] in class 'Utils' at line 37, char 4: Type mismatch: NUMBER expected for variable oops1
Error [Concatenation] in class 'Utils' at line 38, char 19: Concatenation only works for strings
Error [Negation] in class 'Utils' at line 39, char 17: Logical negation requires BOOL
Error [PowerContext] in class 'Utils' at line 40, char 19: Power only works for numbers
Error [FuncCall] in class 'Utils' at line 41, char 19: Argument count mismatch in call to compute
Error [FuncCall] in class 'Utils' at line 42, char 19: Argument count mismatch in call to compute
Error [IfStatement] in class 'Main' at line 60, char 4: If condition must be BOOL
Error [ConjunctionContext] in class 'Main' at line 62, char 13: Logical AND requires BOOLs
Error [DivisionContext] in class 'Main' at line 63, char 15: Division by zero error

Error report file location: index.html
```

HTML output of full_test_suite2.txt:

Semantic Errors

AdditionContext: Addition only works for numbers
Class Utils, Line: 37, Char: 17

PrimitiveValueDect: Type mismatch: NUMBER expected for variable oops1
Class Utils, Line: 37, Char: 4

Concatenation: Concatenation only works for strings
Class Utils, Line: 38, Char: 19

Negation: Logical negation requires BOOL
Class Utils, Line: 39, Char: 17

PowerContext: Power only works for numbers
Class Utils, Line: 40, Char: 19

FuncCall: Argument count mismatch in call to compute
Class Utils, Line: 41, Char: 19

FuncCall: Argument count mismatch in call to compute
Class Utils, Line: 42, Char: 19

IfStatement: If condition must be BOOL
Class Main, Line: 60, Char: 4

ConjunctionContext: Logical AND requires BOOLs
Class Main, Line: 62, Char: 13

DivisionContext: Division by zero error
Class Main, Line: 63, Char: 19

Class: Utils

Variables	Functions
NUMBER val	NUMBER compute(NUMBER, NUMBER)
BOOL valid	BOOL logicCheck(BOOL, BOOL)
BOOL oops3	STRING greet(STRING)
NUMBER oops4	
NUMBER oops5	
STRING[] names	
NUMBER oops6	
NUMBER[][] data	
NUMBER crazy	
NUMBER oops1	
STRING[] empty	
STRING oops2	

Class: Main (inherits Utils)

Variables	Functions
BOOL b	None
NUMBER result	
NUMBER x	
NUMBER y	
NUMBER z	

Source Code

```

1 class Utils {
2     // Basic math function using multiple expressions
3     fun compute(x: NUMBER, y: NUMBER): NUMBER {
4         return (x + y) * (x - y) / (x * y) - 2
5     }
6
7     // Logical chaining and implication
8     fun logicCheck(a: BOOL, b: BOOL): BOOL {
9         return (a and b) or not a -> b
10    }
11
12    // String manipulation using concat
13    fun greet(name: STRING): STRING {
14        return "Hello, " + name + "!"
15    }
16 }

```

Both options are provided so user can choose one that fits their desires, whether for debugging purpose or convenience.

5.2. Multi-File Input Support

The compiler now supports multiple input files in a single run.

- All specified source files are read in the order given.
- Their contents are concatenated without separation.
- The resulting combined source is passed through parsing and type checking as one unit.

Example Usage (both features):

```
Java -jar my-compiler.jar file1.txt file2.txt --html --console
```

This example checks both files as one program, prints errors to the console, and generates an HTML report.

5.3. Semantic Error Reporting

The compiler reports semantic errors with detailed information, such as:

- Error type (e.g., Type Mismatch, Undefined Variable, Argument Count Mismatch)

- The class or function in which the error occurs
- Exact line and character position
- Clear error messages that aid debugging

Furthermore, if the output is generated by the **-html** flag, the reports will provide a user-friendly summary with color-coded annotations for easier review. Source code is also pasted under the error reporting so users don't have to go back and forth from html form to the IDE.

Example: take full_test_suite2.txt as input file

Error [AdditionContext] in class 'Utils' at line 37, char 17: Addition only works for numbers

Error [PrimitiveInitDecl] in class 'Utils' at line 37, char 4: Type mismatch: NUMBER expected for variable oops1

Error [Concatenation] in class 'Utils' at line 38, char 19: Concatenation only works for strings

=> Error reporting include:

- **Error type:** e.g., Type Mismatch, Undefined Variable, Argument Count Mismatch
- **Location:** The class or function where the error occurs
- **Position:** Exact line and character number in the source code
- **Message:** Clear descriptions that aid debugging

5.4. Extensible Architecture

The compiler is implemented using a modular, object-oriented design:

- Lexer and parser components (ANTLR-based)
- Separate type checker and visitor class
- Pluggable error listener and reporter modules

This structure makes it easy to extend the compiler with additional language features (e.g., inheritance, lambda expressions, new types) or support new output formats.

6 Limitations

While the compiler successfully performs type-checking on a wide range of object-oriented programs, there are still limitations due to design trade-offs, time constraint of the project, making the compiler not compatible for large scale projects.

6.1. One pass compilation with upward visibility only

The compiler performs semantic checking in a single pass, meaning that it only processes the source code linearly from top to bottom, without going back and forth to re-analyze earlier parts of the code. This design simplifies the implementation and makes the compiler lightweight, but introduces a lot of important limitations such as:

- No forward references: Symbols such as **functions, methods, variables, or classes** must be declared **before they are used** in the code. For example:

```
printGreeting();    // Error: function not yet declared
```

```
func printGreeting() {  
    print("Hello");  
}
```

- If programmers use this compiler, they will have to record their code so that all methods, variables and classes are declared before any use. This restricted the freedom and flow of creativity in programs, especially for users who are used to multi-pass compilers (like Java or C++).
- If the codebase is large-scale, then this feature is a huge obstacle as it is grader to organize naturally

6.2. Limited datatypes

The current version implements basic but functional data types, which form a necessary system that support only the following primitive data types:

- **NUMBER**: for representing integer
- **BOOLEAN**: logical values
- **STRING**: textual data

These types cover most fundamental programming constructs, and sufficient enough for expressing wide range of operations and control structures in programs.

The compiler does not support advanced types such as:

- Integer values or floating point numbers should be declared separated like Java compiler, like **int** and **float**.
- **Enumerations** (e.g., enum Color { Red, Green, Blue })
- **Generic Types** (e.g., List<T>, Map<K, V>)
-
- **Structs** (e.g., struct Point { x: number, y: number })
- **Union Types or Optional Types** (e.g., number | string, Optional<number>)

Though this design decision keeps the compiler architecture simpler and focuses on core type-checking functionality, it limits the ability to scale the language toward more advanced programming.

6.3. Restricted Expression Handling in chains

The compiler does not support direct chaining of expressions which include array indexing followed by method call or field accesses, for example:

```
array[0].doSthPls();  
array[1].fields;
```

This code snippet is not allowed in the compiler, and must be broken into multiple steps, like creating an intermediate variable to hold the value of the array index, and then call the method/access the field from that variable, such as:

```
let temp = array[0];  
temp.doSomething();  
  
let temp2 = array[1];  
let val = temp2.field;  
//these are allowed
```

This restriction because the semantic analyzer of the compiler processes expressions in a single-pass manner. Handling deeply nested or chained expression would require more complex scope and type resolution logic, which is not implemented in the current design

6.4. No support for function overloading

The compiler does not support method overloading, which is defining multiple functions with the same name but different parameter types. This feature is embedded in many languages like Java.

```
fun add(x: number, y: number): number { ... }  
fun add(x: string, y: string): string { ... } // This is not supported
```

All function names must be unique, regardless of their signature, which might be troublesome for projects with larger scale.

6.5. No support for Anonymous Functions or Lambdas

There is currently no support for inline functions, such as lambdas or anonymous functions. For instance:

```
let f = (x: number) => x * x; // This is not supported
```

Only named, top-level functions or methods are supported.

6.7. Little standard library or built-in methods

The programming language which is supported by the compiler have little standard library, which means there are not enough utility functions for frequently performed tasks such as

- **Mathematical functions (e.g., `sqrt()`, `pow()`, `abs()`)**
- **String manipulation (e.g., `substring()`, `toUpperCase()`, `replace()`)**
- **Array functions (e.g., `map()`, `filter()`, `sort()`)**

Though the compiler already supported functions like: `append()`, `insert()`, `print()`, etc, which are qualified as built-in methods, these are not enough to fully support more complex structured projects.

This will force all users to implement these functions from scratch, which will cost a lot of time and development efforts, particularly for beginners or those expecting a higher-level language experience like Java or Python. Furthermore, this can make the language less practical for writing real-world applications, unless the missing functionality is added manually.

6.8. Limited Control Flow Support

While basic `if`, `else`, and `while` statements are supported, there is:

- **No `switch/case` statements**
- **No `break` or `continue` statements within loops**

This limitation means users must rewrite more complex logic using nested `if/else` and `while` loops, which can reduce code clarity.

CONTRIBUTIONS:

Charles Reed	Core Infrastructure • Designed and implemented the static type checker's architecture using the visitor pattern. Basic Type Checking • Implemented logic to create and assign primitive type variables. • Built logic for variable-to-value type checking. • Handled undeclared variables, invalid assignments, and type mismatches. • Added type checking support for comparative, arithmetic, and logical operations. Conditional Statements
--------------	--

	• Added support for if-else statements for control flow along with type checking to ensure booleans are used Object-Oriented Programming Support • Added support for class-based OOP: ○ Class declarations and constructors ○ Dot notation for accessing fields and methods ○ Method resolution and type validation Inheritance and Polymorphism • Enabled single inheritance with correct type hierarchy resolution. • Supported polymorphic method calls and subtype compatibility checking. Array Support • Supported array declarations, indexing, and assignments. • Implemented type checking for polymorphic arrays and nested subarray structure validation. Error Reporting System • Built the Error Reporting Manager for centralized error tracking and formatting. • Developed the HTML Reporter for readable, styled semantic error output. Test Suite Development • Created a comprehensive test suite covering variable declarations, expressions, control flow, function calls, object access, arrays, and inheritance. • Generated expected outputs (HTML and console) for all test cases. Input Handling and CLI Support • Added support for multiple source files to be compiled in a single run. • Implemented command-line flags: ○ <code>--html</code>: generates HTML error reports ○ <code>--console</code>: prints semantic errors to the console ○ Both can be combined or omitted Final Project Report • Wrote Part 3: Justification of Output, explaining the logic behind semantic error detection and reporting. • Co-authored Part 5: Reflections and Contributions, documenting individual responsibilities and team
--	--

	collaboration.
Prantik Roy	Compiler - Came up with and made the design for all the primitive data types (NUMBER, BOOL, STRING) - Implemented the deceleration of primitive type variables - Implemented the creation and assignment of primitive type expressions - Came up with and made the implementation for all the arithmetic, logic, & relational operators, including string operators like concatenation - Created the initial design and implementation of array data types and their literals (i.e. [1, 2, 3], [true], ["a", "ab"], ...) - Designed the compiler to stop when any syntax error was detected Report - Completed Section 1.1 of the report called "Structure of a Program" - Completed Section 1.2 of the report called "List of Advanced Programming Features" - Completed Section 2 of the report called "Output Structure"
Quang Tran	Final Project Report • Completed Section 4: Summary of examples • Completed Section 5: Miscellaneous Features • Completed Section 6: Limitations Compiler Development • Implemented support for functions, including declaration parsing, semantic checks for argument matching, and return type validation. • Revising ANTLR grammar file to support function implementation • Contributed to the semantic error reporting system, ensuring accurate messages for type mismatches, undefined symbols, and function invocation error Version Control • Managed project files via GitHub.